

CNN10 入試対策アプリ

NFT 認証ログイン 完全構築案

エピスミントサイト NFT 所有者限定アクセス制御

最終更新: 2026-04-05

概 要

エピスミントサイト (Polygon NFT) で発行された特定の NFT を所有する生徒のみが、本アプリのコンテンツを閲覧できるようにする認証システムの構築案。

方式: Firebase Authentication + Epis API 連携
生徒のウォレット操作不要 / 既存アカウントでログイン可能

最終更新: 2026-04-05

ステータス: 設計完了 / 実装待ち

目次

1. プロジェクト概要
2. 現状分析
3. エピスミントサイト技術調査結果 (ソースコード解析)
4. 方式選定
5. 全体アーキテクチャ
6. 認証フロー詳細
7. 画面構成
8. 技術スタック
9. ファイル構成
10. 環境変数の設定
11. 実装手順 (Phase 1〜5)
12. 各ファイルの実装仕様
13. セキュリティ設計
14. 運用フロー
15. デプロイ手順
16. 実装開始前に必要な情報

1. プロジェクト概要

目的

エピスミントサイト (<https://app.epis-mintsite.com>) の ユーザーID とパスワード でログインし、特定の NFT を所有する生徒のみが CNN10 入試対策アプリの記事を閲覧できるようにする。

ゴール

エピスミントサイトのユーザーID + パスワードでログイン

→ NFT所有確認

→ 所有者のみ記事閲覧可能

NFTを持たないユーザー

→ ログインはできるが記事閲覧不可 (/no-access にリダイレクト)

想定ユーザー

ユーザー	操作
------	----

生徒 (NFT所有者)	エピスミントサイトと同じユーザーID + パスワードでログイン -> 記事閲覧
管理者 (塾運営者)	エピスミントサイトで NFT を発行/回収してアクセス権を制御
一般訪問者	ログイン画面のみ表示。記事は閲覧不可

2. 現状分析

CNN10 入試対策アプリ (本アプリ)

項目	現状
フレームワーク	Next.js 16.2.1 + React 19.2.4
スタイリング	Tailwind CSS 4
コンテンツ管理	Markdown ファイル (content/YYYY-MM-DD.md)
認証	なし (全ページ公開)
API ルート	なし
ミドルウェア	なし
デプロイ	Vercel

現在のページ構成

パス	内容	ファイル
/	最新記事 + 過去記事リンク	src/app/page.tsx
/daily/[date]	日付別記事詳細	src/app/daily/[date]/page.tsx
/archive	月別バックナンバー一覧	src/app/archive/page.tsx

現在の依存パッケージ

```
{
  "dependencies": {
    "next": "16.2.1",
    "react": "19.2.4",
    "react-dom": "19.2.4",
    "react-markdown": "^10.1.0",
    "remark-gfm": "^4.0.1"
  }
}
```

3. エピスミントサイト技術調査結果 (ソースコード解析)

GitHub リポジトリ (<https://github.com/epis-mintsite/epismintsite>) のソースコード解析結果。

プラットフォーム概要

項目	内容
ブロックチェーン	Polygon (Alchemy RPC 使用)
NFT コントラクト	カスタム (EpisMintSiteNFT)
認証	Firebase Authentication (Email/Password)
バックエンド	NestJS (TypeScript) 、ポート 5000
フロントエンド	React + Vite (Orval で API クライアント自動生成)
コントラクトライブラリ	ethers.js v6
メタデータ保存	Pinata (IPFS)
DB	PostgreSQL (TypeORM)

ログイン画面の構成 (Login.tsx 解析結果)

ファイル: frontend/src/usecases/auth/pages/Login.tsx

フォーム要素	ラベル	バリデーション
ユーザーID	ユーザーID	英数字のみ、スペース不可
パスワード	パスワード	6文字以上、スペース不可
送信ボタン	ログイン	
リンク	アカウントを作成する -> /signup	

認証の仕組み (重要な発見)

ユーザーIDはメールアドレスではない。 エピスミントサイトでは、任意の英数字のユーザーIDを Firebase Auth に {userId}@example.com の形式で登録している。

```
【サインアップ時】 ( Auth.tsx / signup関数 )
createUserWithEmailAndPassword(auth, `${userId}@example.com`, password)

【ログイン時】 ( Login.tsx / handleLogin関数 )
login(userId + '@example.com', password)
→ signInWithEmailAndPassword(auth, email, password)
```

つまり:

- ユーザーID taro123 -> Firebase には taro123@example.com で登録
- ログイン時も taro123@example.com + パスワードで認証
- 生徒はメールアドレスを使わず、ユーザーIDとパスワードだけでログインする

バックエンド認証フロー (firebase.strategy.ts 解析結果)

1. フロントエンドが Firebase ID Token を Bearer ヘッダーで送信
2. passport-firebase-jwt が Token を抽出
3. AuthService.validateFirebaseToken() で firebase-admin により Token を検証
 - decodedToken.uid を取得
 - admin.auth().getUser(uid) でユーザー情報を取得
4. FirebaseStrategy.validate() で DB の users テーブルを照会
 - user.uid = decodedToken.uid でユーザーを検索
 - userType リレーションも JOIN して取得
5. User エンティティを返却 → リクエストに @User() として注入

User エンティティ構造 (user.entity.ts 解析結果)

```
@Entity('users')
class User {
  @PrimaryColumn({ name: 'userId' })
  userId: string;      // ユーザーID ( 例: "taro123" )

  @Column({ length: 255 })
  nickname: string;    // ニックネーム

  @Column({ name: 'imageUrl', type: 'text', nullable: true })
  imageUrl: string;    // プロフィール画像

  @Column({ name: 'isBanned', default: false })
  isBanned: boolean;   // BAN フラグ

  @Column({ name: 'isAdmin', default: false })
  isAdmin: boolean;    // 管理者フラグ

  @Column({ name: 'isSuperAdmin', default: false })
  isSuperAdmin: boolean; // スーパー管理者フラグ

  @Column({ length: 255 })
  address: string;     // ウォレットアドレス

  @Column({ name: 'encryptedPrivateKey', length: 255 })
  encryptedPrivateKey: string; // 暗号化された秘密鍵

  @Column({ length: 255 })
  uid: string;         // Firebase UID ( ★ 認証のキー )

  @Column({ name: 'lastLoggedInAt', nullable: true })
  lastLoggedInAt: Date; // 最終ログイン日時

  // リレーション
  userType: UserType;   // ユーザー種別
  classroom: Classroom; // 教室
  grade: Grade;         // 学年
  userNfts: UserNft[];  // 所有NFT一覧 ★
  userCoins: UserCoin[]; // コイン
  // ... その他
}
```

NFT 関連の API エンドポイント

エンドポイント	認証	内容
GET /users/accessible-nfts	Bearer Token 必須	ユーザーがアクセス可能な NFT 一覧
GET /nfts	Bearer Token 必須	全 NFT 一覧
GET /nfts/:nftId	不要	特定 NFT の詳細
GET /nfts/public/all	不要	公開 NFT 一覧

POST /nfts/mint	Bearer Token 必須	NFT ミント (発行)
POST /nfts/burn	Bearer Token 必須	NFT バーン (焼却)

NFT データ構造 (UserNftItemDto)

```
{
  nftId: string;    // NFT ID ( ★ この値で所有判定する )
  ownerAddress: string; // 所有者のウォレットアドレス
  ownerId: string;  // 所有者のユーザーID
  title: string;    // NFT名
  imageUrl: string; // 画像URI ( IPFS )
  description: string; // 説明
  tags: string[];   // タグ
  isPublic: boolean; // 公開フラグ
  createdAt: string; // 作成日時
}
```

useFirebaseAuth フック (フロントエンド認証管理)

```
- onAuthStateChanged でログイン状態を監視
- user.getIdToken() で Firebase ID Token を取得
- 25分ごとに Token を自動更新 ( setInterval )
- タブがアクティブに🔒🔒したときも Token を再取得
- エラー時は /login にリダイレクト
- accessToken を state で管理し、API 呼び出し時に使用
```

4. 方式選定

候補の比較

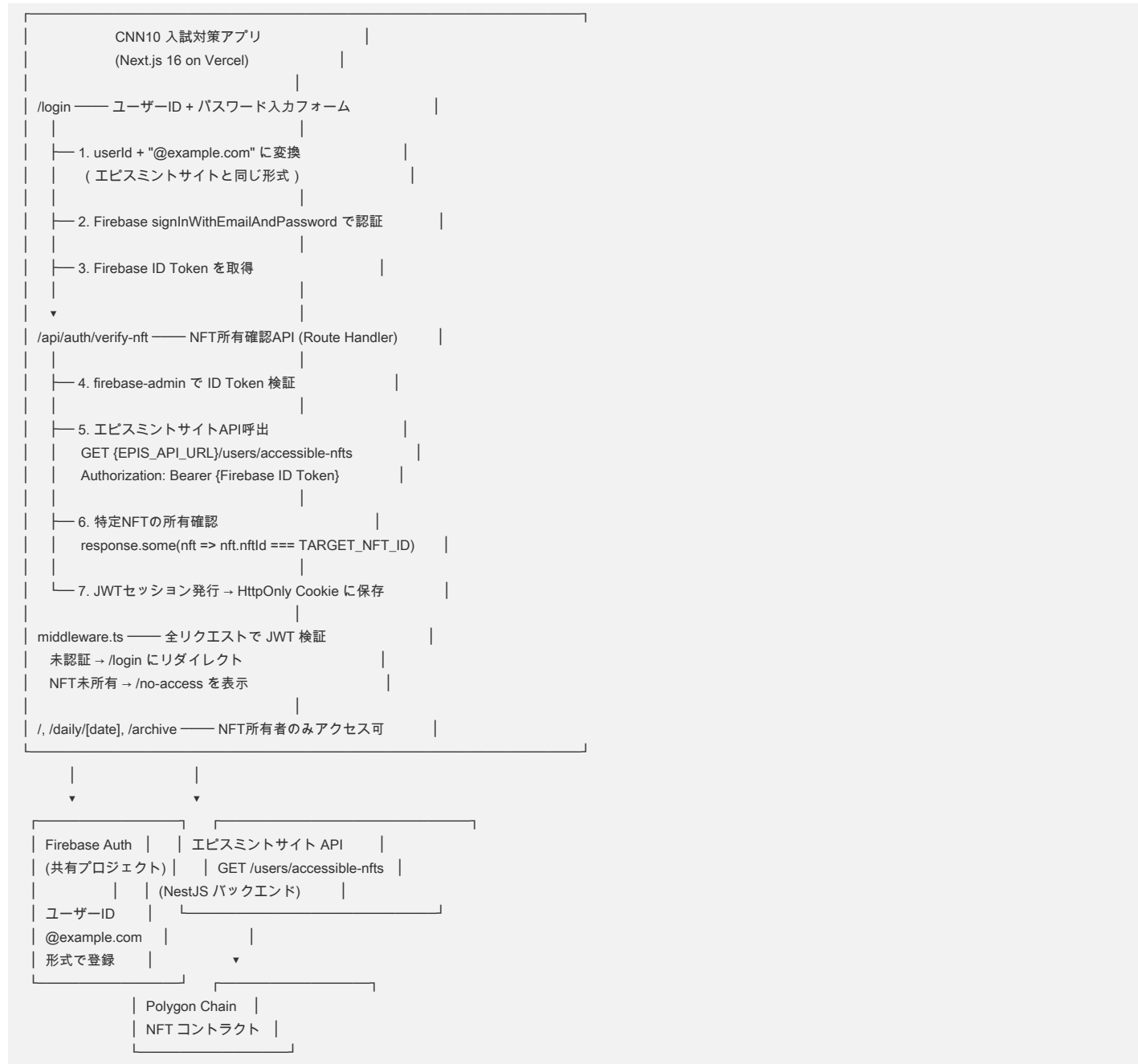
方式	概要	メリット	デメリット	判定
A: Firebase + Epis API 連携	同じプロジェクトを共有し、NF T 確認は Epis API 経由	Firebase 既存認証の再利用、生徒は同じユーザーID/パスワードでログイン可能、ウォレット操作不要	Epis API への依存	採用
B: ウォレット接続 + コントラクト直接照会	生徒が MetaMask でウォレット接続	MetaMask 非依存	MetaMask 必須、中学生にはUXが複雑	不採用
C: 独自認証 + コントラクト照会	独自の認証基盤を構築	完全独立	構築コスト大、生徒の二重登録が必要	不採用

方式 A を採用する理由

- 1. 生徒の負担ゼロ: エピスミントサイトと同じ ユーザーID + パスワード でログインできる
- 2. ウォレット操作不要: 中学生が MetaMask を使う必要がない
- 3. 実装コスト最小: Firebase 設定の共有 + API 1本の呼び出しで実現

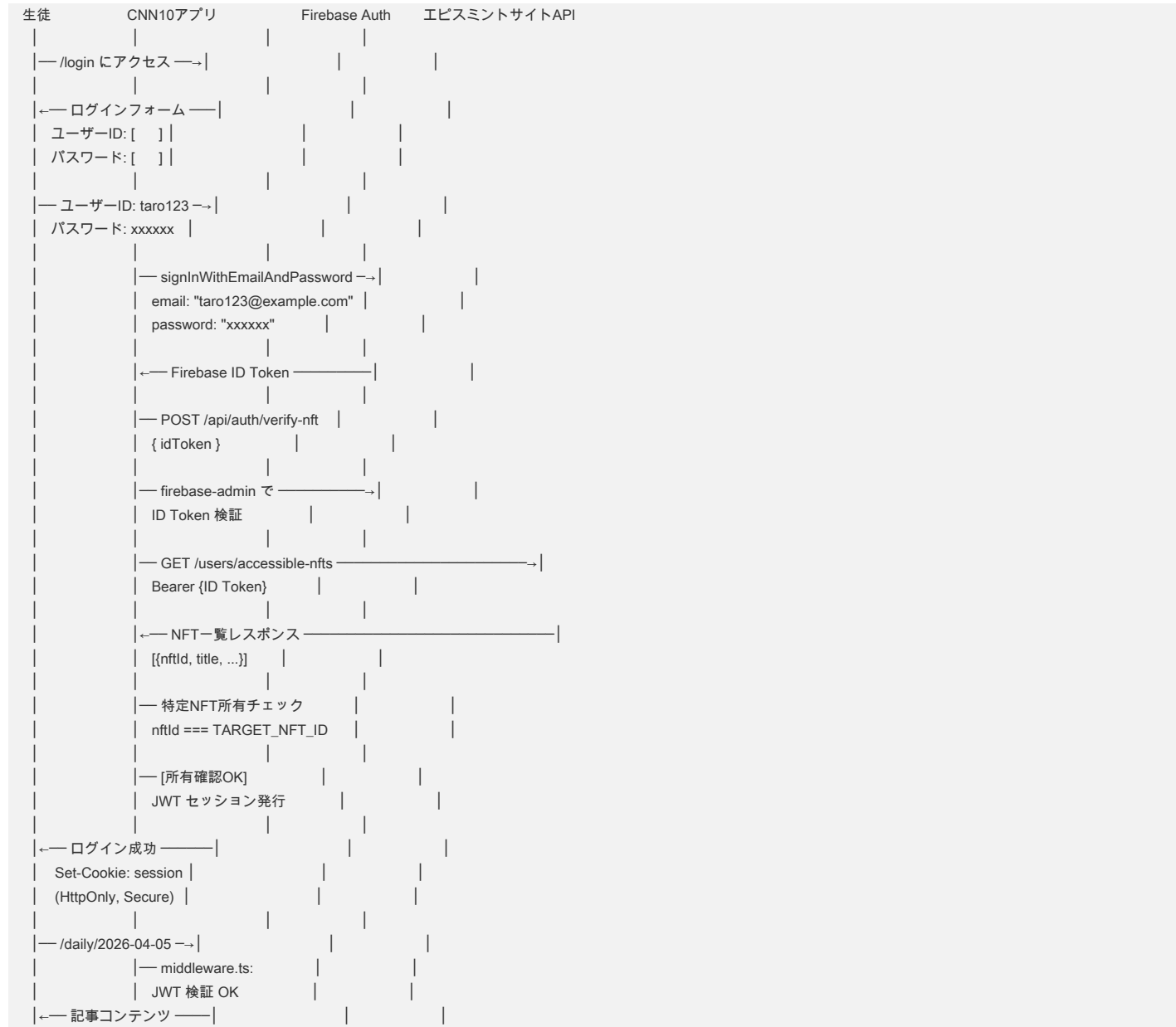
4. 管理が簡単: NFT の発行/回収だけでアクセス権を制御できる

5. 全体アーキテクチャ



6. 認証フロー詳細

シーケンス図



ポイント: ユーザーID -> Firebase メール変換

```
// CNN10 アプリのログイン処理 ( エビスミントサイトと同じ変換ルール )
const firebaseEmail = `${userId}@example.com`;
await signInWithEmailAndPassword(auth, firebaseEmail, password);
```

エビスミントサイトの Login.tsx と Auth.tsx のソースコードから確認した、完全に同じ変換ルールを使用する。

セッション管理

項目	設定
トークン形式	JWT (jose ライブラリ)
保存先	HttpOnly Cookie (session)
有効期限	24時間 (毎日再ログインで NFT 所有状態を再検証)
Cookie 属性	HttpOnly, Secure, SameSite=Lax, Path=/

7. 画面構成

画面	パス	アクセス権限	内容
ログイン	/login	全員	ユーザーID + パスワード入力 (エピスミントサイ トと同じ)
NFT未所有	/no-access	ログイン済み・ NFT未所有	「NFTを取得してください」案内ペ ージ
最新記事	/	NFT 所有者のみ	最新の CNN10 分析 + 過去記事リンク
記事詳細	/daily/[date]	NFT 所有者のみ	各日の CNN10 分析コンテンツ
バックナンバー	/archive	NFT 所有者のみ	月別アーカイブ一覧

ログイン画面のUI仕様

Daily CNN 10 Exam Prep ← ヘッダー

ログイン

ユーザーID

アルファベット・数字で記入

パスワード

6文字以上で記入

ログイン

※ エピスミントサイトと
同じID・パスワード◆◆使用

バリデーション (エピスミントサイトと同じルール)

フィールド	ルール	エラーメッセージ
ユーザーID	スペース不可	「スペースは使用できません」
ユーザーID	英数字のみ (/^[a-zA-Z0-9]+\$ /)	「アルファベットと数字のみ使用可能です」
パスワード	スペース不可	「スペースは使用できません」
パスワード	6文字以上	「6文字以上で記入してください」

8. 技術スタック

追加パッケージ

パッケージ	役割	インストール
firebase	フロントエンド Firebase SDK (signInWithEmailAndPassword)	Auth npm install firebase
firebase-admin	サーバー側 Firebase ID Token 検証	npm install firebase-admin
jose	JWT 生成・検証 (Edge Runtime 対応セッション管理)	npm install jose

9. ファイル構成

追加・変更するファイル一覧

```

my-next-app/
├── src/
│   ├── app/
│   │   ├── login/
│   │   │   └── page.tsx      ← 【新規】ログインページ ( ユーザーID + パスワード )
│   │   ├── no-access/
│   │   │   └── page.tsx      ← 【新規】NFT未所有ページ
│   │   ├── api/
│   │   │   ├── auth/
│   │   │   │   ├── verify-nft/
│   │   │   │   │   └── route.ts      ← 【新規】NFT検証 API
│   │   │   │   └── logout/
│   │   │   │       └── route.ts      ← 【新規】ログアウト API
│   │   ├── layout.tsx      ← 【変更】ヘッダーにログアウトボタン追加
│   │   └── page.tsx        ← 変更なし
│   ├── components/
│   │   ├── MarkdownContent.tsx    ← 変更なし
│   │   └── LogoutButton.tsx      ← 【新規】ログアウトボタン ( Client Component )
│   ├── lib/
│   │   ├── firebase.ts          ← 【新規】Firebase Client SDK 初期化
│   │   ├── firebase-admin.ts    ← 【新規】Firebase Admin SDK 初期化
│   │   ├── session.ts           ← 【新規】JWT セッション管理ユーティリティ
│   │   ├── daily.ts             ← 変更なし
│   │   └── middleware.ts        ← 【新規】認証ミドルウェア
│   ├── .env.local              ← 【新規】環境変数 ( ローカル )
│   └── docs/
│       └── nft-auth-plan.md      ← 本ドキュメント

```

10. 環境変数の設定

.env.local に設定する値

```
# ===== Firebase クライアント設定 =====
# エピスミントサイトと同じ Firebase プロジェクトの値を使用
# Firebase コンソール → プロジェクトの設定 → 全般 → マイアプリ から取得
NEXT_PUBLIC_FIREBASE_API_KEY="..."
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN="..."
NEXT_PUBLIC_FIREBASE_PROJECT_ID="..."
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET="..."
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID="..."
NEXT_PUBLIC_FIREBASE_APP_ID="..."

# ===== Firebase Admin SDK =====
# Firebase コンソール → プロジェクトの設定 → サービスアカウント → 秘密鍵の生成
FIREBASE_PROJECT_ID="..."
FIREBASE_CLIENT_EMAIL="firebase-adminsdk-xxxxx@your-project.iam.gserviceaccount.com"
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----"

# ===== エピスミントサイト API =====
# エピスミントサイトのバックエンド API URL
EPIS_API_URL="https://api.epis-mintsite.com"

# ===== NFT 認証 =====
# CNN10アプリへのアクセスを許可するNFTのID
TARGET_NFT_ID="対象のNFT ID"

# ===== セッション =====
# openssl rand -base64 32 で生成
SESSION_SECRET="ランダムな32文字以上の文字列"
```

11. 実装手順 (Phase 1〜5)

Phase 1: Firebase Auth セットアップ (ユーザーID + パスワードログイン)

手順:

1. パッケージインストール

```
npm install firebase firebase-admin jose
```

2. src/lib/firebase.ts — Firebase Client SDK 初期化

```
import { initializeApp, getApps } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = getApps().length === 0
  ? initializeApp(firebaseConfig)
  : getApps()[0];

export const auth = getAuth(app);
```

3. src/lib/firebase-admin.ts — Firebase Admin SDK 初期化

```
import { initializeApp, getApps, cert } from "firebase-admin/app";
import { getAuth } from "firebase-admin/auth";

const app = getApps().length === 0
  ? initializeApp({
    credential: cert({
      projectId: process.env.FIREBASE_PROJECT_ID,
      clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
      privateKey: process.env.FIREBASE_PRIVATE_KEY?.replace(/\n/g, "\n"),
    }),
  })
  : getApps()[0];

export const adminAuth = getAuth(app);
```

4. src/app/login/page.tsx — ログインページ (ユーザーID + パスワード)

```

"use client";
import { useState, FormEvent } from "react";
import { signInWithEmailAndPassword } from "firebase/auth";
import { auth } from "@/lib/firebase";
import { useRouter } from "next/navigation";

export default function LoginPage() {
  const [userId, setUserId] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  // バリデーション ( エビミントサイトと同じルール )
  const validate = (): string | null => {
    if (/^/.test(userId)) return "スペースは使用できません";
    if (!/^[a-zA-Z0-9]+$/.test(userId))
      return "アルファベットと数字のみ使用可能です";
    if (/^/.test(password)) return "スペースは使用できません";
    if (password.length < 6) return "6文字以上で記入してください";
    return null;
  };

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();
    const validationError = validate();
    if (validationError) { setError(validationError); return; }

    setLoading(true);
    setError("");

    try {
      // ★ エビミントサイトと同じ変換: userId → userId@example.com
      const firebaseEmail = `${userId}@example.com`;
      const credential = await signInWithEmailAndPassword(
        auth, firebaseEmail, password
      );
      const idToken = await credential.user.getIdToken();

      // NFT 所有確認 API を呼び出し
      const res = await fetch("/api/auth/verify-nft", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ idToken }),
      });

      if (res.ok) {
        router.push("/");
      } else {
        const data = await res.json();
        if (res.status === 403) {
          router.push("/no-access");
        } else {
          setError(data.error || "ログインに失敗しました");
        }
      }
    } catch {
      setError("ログインに失敗しました");
    } finally {
      setLoading(false);
    }
  };

  return (/* ログインフォーム UI */);
}

```

Phase 2: NFT 所有検証 API

src/app/api/auth/verify-nft/route.ts:

```
import { NextResponse } from "next/server";
import { adminAuth } from "@lib/firebase-admin";
import { createSession } from "@lib/session";

const EPIS_API_URL = process.env.EPIS_API_URL!;
const TARGET_NFT_ID = process.env.TARGET_NFT_ID!;

export async function POST(request: Request) {
  const { idToken } = await request.json();

  // 1. Firebase ID Token 検証
  const decodedToken = await adminAuth.verifyIdToken(idToken);

  // 2. エピスメントサイト API で NFT 一覧取得
  const res = await fetch(`${EPIS_API_URL}/users/accessible-nfts`, {
    headers: { Authorization: `Bearer ${idToken}` },
  });
  const nfts = await res.json();

  // 3. 対象 NFT の所有確認
  const hasAccess = nfts.some(
    (nft: { nftId: string }) => nft.nftId === TARGET_NFT_ID
  );

  if (!hasAccess) {
    return NextResponse.json({ error: "NFT not found" }, { status: 403 });
  }

  // 4. JWT セッション発行
  const session = await createSession({
    uid: decodedToken.uid,
    email: decodedToken.email,
  });

  // 5. Cookie に保存
  const response = NextResponse.json({ success: true });
  response.cookies.set("session", session, {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    maxAge: 60 * 60 * 24, // 24時間
    path: "/",
  });

  return response;
}
```

Phase 3: JWT セッション管理 + Middleware

src/lib/session.ts:

```
import { SignJWT, jwtVerify } from "jose";

const secret = new TextEncoder().encode(process.env.SESSION_SECRET);

export async function createSession(payload: { uid: string; email?: string }) {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("24h")
    .setIssuedAt()
    .sign(secret);
}

export async function verifySession(token: string) {
  try {
    const { payload } = await jwtVerify(token, secret);
    return payload;
  } catch {
    return null;
  }
}
```

src/middleware.ts:

```
import { NextRequest, NextResponse } from "next/server";
import { verifySession } from "@lib/session";

const PUBLIC_PATHS = ["/login", "/no-access", "/api/auth/"];

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  if (PUBLIC_PATHS.some((p) => pathname.startsWith(p))) {
    return NextResponse.next();
  }

  if (pathname.startsWith("/_next") || pathname.includes(".")) {
    return NextResponse.next();
  }

  const session = request.cookies.get("session")?.value;
  if (!session) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  const payload = await verifySession(session);
  if (!payload) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/(?!(?!_next/static|_next/image|favicon.ico).*)"],
};
```

Phase 4: 既存ページの保護 + UI 変更

1. src/app/no-access/page.tsx — NFT 未所有ページ
2. src/components/LogoutButton.tsx — ログアウトボタン
3. src/app/api/auth/logout/route.ts — セッション Cookie 削除
4. src/app/layout.tsx — ヘッダーにログアウトボタン追加

Phase 5: Vercel デプロイ

Vercel ダッシュボードで全環境変数を設定してデプロイ。

12. 各ファイルの実装仕様

src/app/login/page.tsx (ログインページ)

項目	内容
ディレクティブ	"use client"
UI要素	ユーザーID入力、パスワード入力、ログインボタン
ユーザーID -> Firebase メール変換	\${userId}@example.com (エピスメントサイトと同じ)
認証処理	signInWithEmailAndPassword -> getIdToken -> POST /api/auth/verify-nft
バリデーション	英数字のみ、スペース不可、パスワード6文字以上
成功時	/ にリダイレクト
NFT未所有時	/no-access にリダイレクト
エラー時	「ログインに失敗しました」表示

src/app/api/auth/verify-nft/route.ts (NFT 検証 API)

項目	内容
メソッド	POST
リクエスト	{ idToken: string }
処理	Token 検証 -> Epis API で NFT 確認 -> Session 発行
成功レスポンス	200 { success: true } + Set-Cookie: session=...
失敗レスポンス	401 (Token 不正) / 403 (NFT 未所有)

13. セキュリティ設計

認証の多層構造

Layer 1: ユーザーID + パスワード入力

→ クライアント◆◆バリデーション (英数字のみ、6文字以上)

Layer 2: Firebase Authentication

→ userId@example.com 形式で Firebase に認証

→ Firebase が提供するセキュリティ (ブルート◆◆オース防止等)

Layer 3: Firebase ID Token 検証 (firebase-admin)

→ CNN10 サーバー側で Token の署名を検証

Layer 4: エピスミントサイト API による NFT 所有確認

→ 同じ Firebase ID Token で Epis バックエンドにも認証

→ Polygon ブロックチェーン上の NFT 所有状態を確認

Layer 5: JWT セッション (jose)

→ HttpOnly Cookie で XSS 攻撃から保護

→ 24時間有効期限で定期的に NFT 所有状態を再検証

14. 運用フロー

新しい生徒にアクセス権を付与する

1. 管理者がエピスミントサイトで生徒にアカウントを作成 (ユーザーID + パスワード)

2. 管理者がエピスミントサイトで CNN10 用 NFT を生徒にミント

3. 生徒が CNN10 アプリの /login にアクセス

4. エピスミントサイトと同じユーザーID + パスワードでログイン

5. NFT 所有が確認され、コンテンツが閲覧可能に

生徒のアクセス権を取り消す

1. 管理者がエピスミントサイトで該当 NFT をバーン (焼却)

2. 生徒の次回ログイン時 (またはセッション期限切れ後)

3. NFT 所有確認に失敗

4. /no-access ページにリダイレクト → コンテンツ閲覧不可に

15. デプロイ手順

Vercel 環境変数設定

変数名	環境	機密度
NEXT_PUBLIC_FIREBASE_API_KEY	All	公開可
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	All	公開可
NEXT_PUBLIC_FIREBASE_PROJECT_ID	All	公開可
	All	公開可
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET		

	All	公開可
NEXT_PUBLIC_FIREBASE_MESSAGING_SEN		
NEXT_PUBLIC_FIREBASE_APP_ID	All	公開可
FIREBASE_PROJECT_ID	Production	機密
FIREBASE_CLIENT_EMAIL	Production	機密
FIREBASE_PRIVATE_KEY	Production	最重要機密
EPIS_API_URL	Production	機密
TARGET_NFT_ID	Production	変更可能性あり
SESSION_SECRET	Production	機密

16. 実装開始前に必要な情報

#	必要な情報	取得方法	ステータス
1	プロジェクト設定 (6項目)	Firebase コンソール -> 全般 -> Firebase マイアプリ	未取得
2	Firebase Admin SDK 秘密鍵 (3項目)	Firebase コンソール -> サービスアカウント -> 秘密鍵生成	未取得
3	エピスミントサイトの API URL	デプロイ環境の VITE_NEST_ENDPOINT の値	未取得
4	対象 NFT の ID	エピスミントサイトで CNN10 用に発行/指定する NFT	未決定

付録: よくある質問

Q: ログイン時にメールアドレスは必要ですか？

A: 不要です。 エピスミントサイトと同じように、ユーザーID (英数字) とパスワードだけでログインできます。内部的には {userId}@example.com の形式で Firebase に送信していますが、生徒はこれを意識する必要はありません。

Q: エピスミントサイトでパスワードを変更したら CNN10 にも反映されますか？

A: はい。 同じ Firebase プロジェクトを共有しているため、パスワード変更は即座に反映されます。

Q: エピスミントサイトの API がダウンしていたらどうなりますか？

A: 新規ログインは失敗しますが、既にセッション Cookie を持っている生徒は24時間以内であれば引き続きアクセスできます。